LEVEL 1

# Chapter 2: System Design Fundamentals

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026

# Chapter 2: System Design Fundamentals

← *Chapter 1: Prerequisites* · *Index* · *Next* → *Chapter 3: Capacity Estimation*

## 2.1 What System Design Actually Is

**Simple explanation:** System design is the practice of deciding how the pieces of a large software system fit together — which components exist, how they talk to each other, where data lives, and how the whole thing keeps working as load grows and things fail. It's an exercise in trade-offs, not in finding "the correct answer," because there almost never is one correct answer — only answers that fit a given set of requirements, constraints, and priorities better or worse.

**Real-world analogy:** Building a house, you don't start by picking paint colors. You start with: how many people live here, what's the budget, is it earthquake zone, how will it grow if the family grows. Then you decide on foundation, structure, plumbing, electrical — the load-bearing decisions that are expensive to change later. System design interviews test whether you think about software the same way: foundation-first, not paint-first (paint = specific library or exact syntax, which almost never matters at this level).

**Why it exists as its own discipline, separate from coding:** A single machine, in isolation, is relatively easy to reason about. The discipline of system design exists because once you have millions of users, terabytes of data, and dozens of engineers shipping code weekly, a design that "just works" on one machine breaks in a dozen new ways — the network fails, disks fill up, two writes race each other, one server dies at 3 AM. System design is the vocabulary and toolkit for reasoning about *that* world.

### HLD vs LLD

This distinction matters because it defines what an interviewer actually wants from you, and confusing them is one of the most common ways candidates waste their 45 minutes.

| | High-Level Design (HLD) | Low-Level Design (LLD) |
|---|---|---|
| Question shape | "Design Uber" / "Design WhatsApp" | "Design a parking lot" / "Design a rate limiter class" / "Design an elevator system" |
| Focus | Components, data flow, scaling, infrastructure | Classes, interfaces, design patterns, object relationships |
| Output | Boxes-and-arrows architecture diagram | UML-ish class diagrams, method signatures |
| Typical duration | 45–60 min | 45–60 min |

| | High-Level Design (HLD) | Low-Level Design (LLD) |
|---|---|---|
| Tools discussed | Databases, caches, queues, load balancers, CDNs | SOLID principles, OOP design patterns (Factory, Strategy, Observer) |
| This roadmap's focus | **Primary focus** (matches your stated goal) | Referenced briefly where relevant, not a core focus |

Your prompt asked specifically for HLD, so that's this roadmap's center of gravity — but be aware that companies like Razorpay, Flipkart, and Amazon sometimes run a dedicated LLD round (parking lot, ATM, elevator, library management system are the classic questions) in addition to HLD. Chapter 27's question bank flags LLD-style questions separately so you know which framework applies.

## Functional vs. Non-Functional Requirements

**Functional requirements (FR)** — what the system *does*. "Users can send messages." "Users can search for restaurants." "A payment can be refunded." These come from the product spec.

**Non-functional requirements (NFR)** — how well the system does it, and under what constraints. Scale, latency, availability, consistency, security, cost. These come from you — asking clarifying questions is how you *derive* them, and the interviewer is watching whether you ask.

> **Interview question:** "Design Instagram." What are the FRs and NFRs?
> **Ideal senior answer:** "Before I list them, let me confirm scope with you — are we designing the full product or focusing on a slice, like the feed and upload path? [wait for answer] Okay, functional requirements: users upload photos/videos, follow other users, see a feed of posts from people they follow, like and comment. Non-functional: given this is Instagram-scale, I'll assume high read-to-write ratio, hundreds of millions of DAU, feed latency under 200ms, eventual consistency is acceptable for likes/comments but the upload path needs strong durability once a user hits 'post.' Does that match what you want me to focus on?"
>
> Notice what's happening: FRs are stated as a list; NFRs are stated as *assumptions you're checking*, not facts you're asserting. That checking-in is what separates a mid-level from a senior answer — see Chapter 14 for the full framework.

---

## 2.2 The "-ilities": Scalability, Availability, Reliability, Durability

These four get confused with each other constantly. Nail the distinction — it's one of the highest-value five minutes you'll spend in this whole roadmap.

| Term | Plain definition | What breaks if you don't have it |
|---|---|---|
| **Scalability** | The system can handle growing load (more users, more data, more traffic) by adding resources | Site falls over during a flash sale or viral moment |
| **Availability** | The system is up and responding *right now* | Users see errors or timeouts even though the underlying design is otherwise fine |
| **Reliability** | The system does the *correct* thing consistently, without silent errors, over time | Payments get double-charged, orders get lost, wrong data returned |
| **Durability** | Once data is written and acknowledged, it survives failures (disk crash, power loss) permanently | You tell the user "order confirmed" and then lose the order |

**Analogy:** A courier company. *Scalability* is being able to hire more drivers as order volume grows. *Availability* is the phone line being answered right now. *Reliability* is the package actually being the one you ordered, not a mixed-up one. *Durability* is that once they've said "picked up," the package genuinely will not get lost even if their warehouse burns down (because it's insured, tracked, replicated across depots).

**Availability is usually expressed in "nines":**

| Availability | Downtime/year | Downtime/month | Typical for |
|---|---|---|---|
| 99% ("two nines") | ~3.65 days | ~7.3 hours | Internal tools |
| 99.9% ("three nines") | ~8.76 hours | ~43.8 min | Standard production SaaS |
| 99.99% ("four nines") | ~52.6 min | ~4.4 min | Payment systems, core APIs |
| 99.999% ("five nines") | ~5.26 min | ~26 sec | Telecom, critical infra |

> **Interview question:** "How do you design for 99.99% availability?"
> **Ideal senior answer:** "Every nine you add roughly 10x's your engineering cost, so first I'd confirm this is actually the right target for the business — a food delivery app's ordering flow probably does need four nines, but an internal analytics dashboard doesn't. Assuming it's justified: eliminate single points of failure with redundancy at every layer — multiple AZs for compute and database, read replicas, load balancer health checks with automatic failover, circuit breakers so one failing dependency doesn't cascade, and a tested runbook/automated failover for the database layer specifically, since that's usually the hardest part to make redundant without sacrificing consistency."
> **Common mistake:** Treating "high availability" as a single knob you turn on, rather than a property that has to be engineered independently at every layer (LB, app, cache, DB, network) — a system is only as available as its least available critical-path component.

**Durability vs. Availability — the trap:** A system can be highly available (always responding) while being non-durable (the data it returns can be lost) — an in-memory cache with no persistence is a good example: always fast to respond, but a restart wipes it. Conversely, a system can be extremely durable (data safely on disk, replicated three ways) but briefly unavailable during a leader election. **They are independent axes, and conflating them is a very common interview mistake.**

---

## 2.3 Performance: Latency, Throughput, and the Trade-off Between Them

**Latency** — how long a single request takes, end to end. Measured in milliseconds. Always discuss latency using **percentiles**, not averages — this is a real interview signal.

- **p50 (median):** half your requests are faster than this.
- **p95 / p99:** the "tail." 1 in 100 (or 20) requests is *at least* this slow.
- Why percentiles beat averages: if 99 requests take 10ms and one takes 5000ms, the average (~59ms) hides a genuinely bad experience that a p99 (5000ms) exposes immediately. At scale, "rare" tail latency happens to *thousands* of real users every minute — this is why companies like Amazon obsess over p99, not average.

**Throughput** — how much work the system does per unit time. Usually **requests per second (RPS)** or **queries per second (QPS)**, sometimes transactions/sec or messages/sec for pipelines.

**The trade-off:** You can often improve throughput by batching (process 100 requests together, amortizing overhead) — but batching *increases* latency for any individual request, because it now waits for the batch to fill. This exact tension (Kafka batch size, DB write batching, network Nagle's algorithm) shows up constantly once you go past the surface level.

> **Interview question:** "Your system does 500 RPS with p99 latency of 800ms. Product wants 5,000 RPS. What do you look at first?"
> **Ideal senior answer:** "First I'd find the bottleneck, not guess — profile the request path. If it's CPU-bound in the app layer, horizontal scaling of stateless app servers behind the load balancer solves it cheaply. If it's the database, that's harder: I'd look at query optimization and indexing first, then read replicas for read-heavy load, then caching for hot reads, and only reach for sharding if writes themselves are the bottleneck, because sharding adds real operational complexity I don't want to pay for prematurely."

---

## 2.4 Consistency, Fault Tolerance, Maintainability, Observability, Security, Cost

**Consistency** — do all readers see the same data at the same time? Deep dive in Chapter 5/6; for now: **strong consistency** means every read reflects the latest write immediately (harder, often slower, needed for account balances). **Eventual consistency** means reads might briefly return stale data, but all replicas converge eventually (easier, faster, fine for a like count or a feed).

**Fault tolerance** — the system keeps functioning (maybe in a degraded mode) when a component fails, rather than failing entirely. This is different from *availability* (which is the outcome you measure) — fault tolerance is the *mechanism* (redundancy, graceful degradation, circuit breakers) that produces availability.

**Maintainability** — how easily engineers can understand, modify, and extend the system without introducing bugs, months or years after it was written. Rarely asked about directly, but it's *why* interviewers push back on unnecessarily complex designs — a design that's clever but unmaintainable is a design that fails this axis even if it technically "works."

**Observability** — can you tell what the system is doing and why, from the outside, without redeploying it? This is logs + metrics + traces (Chapter 11). It's the difference between "the API is slow" and "the API is slow because the `orders` table's `user_id` index was dropped in yesterday's migration, visible in the trace as a full table scan."

**Security** — protecting data and access from unauthorized use (Chapter 10). Non-negotiable in fintech-adjacent designs; you'll lose serious points at Razorpay/PhonePe-style interviews if you never mention it.

**Cost** — every architecture decision has a dollar cost, and senior/staff candidates are expected to reason about it unprompted. "I'd use DynamoDB here for its predictable scaling, though at very high sustained throughput I'd model on-demand vs. provisioned capacity cost before committing" is a staff-level sentence; never mentioning cost at all is a mid-level signal.

---

## 2.5 CAP Theorem and PACELC

**CAP Theorem** is probably the single most name-dropped — and most misunderstood — concept in system design interviews. Here's the precise version:

In a **distributed system that experiences a network partition** (a break in communication between nodes — and at scale, this *will* happen), you must choose between:

- **Consistency (C):** every node sees the same data at the same time — a read always returns the most recent write, or an error.
- **Availability (A):** every request gets a (non-error) response, even if it might not be the most recent data.

You cannot have both **during a partition**. You can always have both when there's no partition — CAP is specifically a statement about partition behavior, which is the part people forget.

**The critical nuance interviewers listen for:** CAP is not "pick 2 of 3 forever." Partition tolerance (P) isn't really optional in a real distributed system — networks *will* fail — so in practice the real choice is **CP vs. AP** during the partition window.

| System | Typical choice | Why |
|---|---|---|
| Traditional RDBMS (single-node) | N/A — CAP doesn't apply without a partition scenario | Not distributed |
| MongoDB (default config), HBase, Zookeeper | **CP** | Prioritizes correctness — banking, config data |
| Cassandra, DynamoDB (default), Riak | **AP** | Prioritizes uptime — social feeds, shopping carts |
| Spanner, CockroachDB | **CP**, but engineered to minimize the availability cost | Uses synchronized clocks (TrueTime) + consensus to make CP practical at global scale |

> **Interview question:** "Would you choose CP or AP for a ride-hailing app's driver location system?"
> **Ideal senior answer:** "AP. If the network partitions, I'd much rather show a slightly stale driver location than show no drivers at all and lose the booking — the cost of staleness (driver icon off by a few seconds) is low, the cost of unavailability (can't book a ride) is high and directly loses revenue. Compare that to the payment/ledger system in the same app, where I'd choose CP without hesitation, because showing a wrong balance is worse than a brief unavailability."

**PACELC** extends CAP to cover the more common, everyday case — because network partitions are actually rare; the trade-off you make *every single request*, partition or not, is more relevant day-to-day:

> **P**artition: choose between **A**vailability and **C**onsistency (this is CAP) — **e**lse (no partition): choose between **L**atency and **C**onsistency.

Even with a perfectly healthy network, if you want strong consistency (e.g., synchronously replicate a write to 3 nodes before acknowledging it), you pay latency for it. If you want low latency, you accept the replica might lag momentarily.

- **PA/EL** systems (Cassandra, DynamoDB): prioritize availability during a partition, latency otherwise.
- **PC/EC** systems (Spanner, traditional RDBMS with sync replication): prioritize consistency always, accepting the latency cost.

> **Why this is the better framework to actually reach for in interviews:** CAP only tells you what happens in the rare partition case. PACELC tells you what your system is optimizing for on a normal Tuesday afternoon, which is what you're actually designing for 99.9% of the time. Bringing up PACELC unprompted is a strong signal you've gone past the Wikipedia-level understanding of CAP.

---

## 2.6 Horizontal vs. Vertical Scaling

**Vertical scaling ("scale up"):** add more resources (CPU, RAM, disk) to a single existing machine. Simple — no architecture change needed — but has a hard ceiling (the biggest instance type that exists) and creates a single point of failure. A single very large AWS instance is still one instance.

**Horizontal scaling ("scale out"):** add more machines and distribute load across them. No practical ceiling, and inherently more fault-tolerant (one machine dying doesn't take down the system) — but requires the application to be designed for it: statelessness (or externalized state), a load balancer, and often a harder data layer story (which machine has which data?).

| | Vertical | Horizontal |
|---|---|---|
| Complexity | Low | Higher (needs LB, statelessness, data partitioning) |
| Ceiling | Hard limit (biggest box available) | Practically unlimited |
| Fault tolerance | Poor (one box = one failure domain) | Good (redundant instances) |
| Cost curve | Non-linear — big boxes cost disproportionately more per unit of capacity | More linear, but adds operational overhead |
| Downtime to scale | Often requires a restart/resize | Usually zero — just add nodes |

> **Interview question:** "Would you scale your primary PostgreSQL database vertically or horizontally?"
> **Ideal senior answer:** "I'd scale vertically first, and for a surprisingly long time — modern managed Postgres (RDS/Aurora) instances handle a lot of load, and vertical scaling is operationally trivial compared to sharding. I'd add read replicas next for read-heavy load, which is horizontal scaling for reads without touching the harder write-scaling problem. I'd only reach for write-sharding once I have concrete evidence — write throughput or storage genuinely exceeding a single primary's ceiling — because sharding introduces cross-shard query complexity, rebalancing operations, and application-level routing logic that I don't want to carry unless the data justifies it."

---

## 2.7 Stateless vs. Stateful Services

**Stateless service:** doesn't retain any client-specific data between requests. Every request carries everything the server needs (or the server fetches it fresh from a shared store). This is what makes horizontal scaling trivial — spin up 50 identical copies, put a load balancer in front, done, any instance can serve any request.

**Stateful service:** holds onto data specific to a client or session across requests (in-memory session data, an open WebSocket connection, a database itself). Scaling these requires more thought — either **sticky sessions** (route the same client to the same instance — fragile, complicates deployments) or **externalizing the state** (move it to Redis/a DB, making the service stateless again from the app's point of view, at the cost of a network hop).

> **Rule of thumb interviewers want to hear:** design your application/API layer to be stateless wherever possible, and push unavoidable state into purpose-built stateful systems (databases, caches, message brokers) that are specifically engineered to handle it well. Don't build accidental statefulness into your app servers.

---

## 2.8 Monolith vs. Modular Monolith vs. Microservices vs. SOA

**Monolith:** the entire application — UI, business logic, data access — is one deployable unit, typically one codebase, often one database.
- *Advantages:* simple to develop, test, deploy, and reason about early on; no network calls between "modules"; easy transactions (one DB).
- *Disadvantages:* the whole app scales as one unit even if only one part needs more capacity; a bug anywhere can take down everything; large teams step on each other in one codebase; slow deploys as the app grows.
- *When to use:* new products, small teams, unclear domain boundaries (which is most startups) — you don't yet know where the seams should be, and premature microservices lock in a *guess* about those seams.

**Modular monolith:** a monolith with enforced internal boundaries — separate modules/packages with clear interfaces between them, but still one deployable unit and often one database (or one DB per module, logically separated). This is the underrated middle ground: you get organizational clarity without operational complexity, and it's a much easier system to split into real microservices later, because the seams already exist in the code.

**Microservices:** independently deployable services, each owning its own data, communicating over the network (sync or async).
- *Advantages:* independent scaling, independent deployment (team A ships without waiting on team B), technology flexibility per service, fault isolation (one service crashing doesn't necessarily take down others).
- *Disadvantages:* massive operational complexity — network calls where function calls used to be, distributed transactions, service discovery, distributed tracing needed just to debug a single user request, eventual consistency headaches, and organizational overhead (you basically need a platform team).
- *When to use:* you have real, proven independent scaling needs, multiple teams that need to ship independently, and — critically — the operational maturity (CI/CD, observability, on-call practices) to run distributed systems. You clearly have this maturity given your Kubernetes/Prometheus/OTel background, so you can speak to it directly in interviews.

**SOA (Service-Oriented Architecture):** the older, coarser-grained ancestor of microservices — services are typically larger, often communicate through a central **Enterprise Service Bus (ESB)**, and tend to share more infrastructure (sometimes even a shared database). Mentioned mostly so you can correctly answer "how is SOA different from microservices" if asked: microservices favor small, independently deployable services with decentralized data and lightweight communication (REST/gRPC/events) over a central broker, and favor "smart endpoints, dumb pipes" versus SOA's "smart pipes."

> **Interview question:** "Would you start a new food-delivery startup's backend as a monolith or microservices?"
> **Ideal senior answer:** "Modular monolith, honestly — even though 'microservices' is often the expected buzzword. At day one you don't yet know the real domain boundaries (is 'delivery tracking' its own service or part of 'orders'? You genuinely don't know until real usage tells you), and premature microservices means paying full distributed-systems tax — network calls, eventual consistency, service discovery — for boundaries you'll probably redraw in six months anyway. I'd build a modular monolith with clean internal boundaries, instrument it well, and extract services only once a specific module has a proven, independent scaling or team-ownership need — for example, if the notifications module starts needing 10x the infrastructure of everything else, that's a real signal to extract it."
> **This is a staff-level answer specifically because it resists the reflex to sound sophisticated by defaulting to microservices** — see Chapter 23 for more on this exact mistake pattern.

---

## Chapter 2 Interview Drill

1. Explain the difference between availability and reliability with an example where a system has one but not the other.
2. State CAP theorem precisely — including the part most people get wrong (it only applies during a partition).
3. What does PACELC add that CAP doesn't?
4. When would you choose vertical scaling over horizontal, given the operational cost difference?
5. Why is statelessness the single most important property for making horizontal scaling easy?
6. Give one concrete reason to *not* start a new product as microservices.

---

*Next → Chapter 3: Capacity Estimation — the back-of-envelope math framework and 10 fully worked examples.*